JustRUA

GPT-6 机器人使用智能体（RUA）的对照研究

JustRUA: A Controlled Study of GPT-6 Robot-Use Agents

RoboDojo 评测集总分

29.7

↓

50.3

无学习策略 · 无任务调参

Zefan Wang, Yufan Dang, …, Xin Cong, Yuan Yao
清华大学 · 华中科技大学 · 北京大学 · ModelBest · 腾讯

arXiv 2611.16009 · 首次提交 2026-11-11

论文解读

一句话：精简 harness + 廉价几何信息，分数从 29.7 到 50.3

评测集总分（111 集，覆盖 42 个任务）

29.7 → 50.3

成功率 22.5% → 43.3%

无学习策略（VLA）
无自我改进循环
无任务特定调参

JustRUA 保留（5 项）

- 完整的 episode 历史（含全部图像）
- 每段动作最多执行 20 个控制步
- 紧凑文本格式的夹爪读数
- Depth Pro 从 RGB 预测的深度图
- 代码从像素求解 3D 坐标

试过但未保留

- 动作接口：相对目标 / 语义技能 / 自选步数 / 动作序列
- 复用经验：视觉演示 / 文本轨迹 / 自建技能库
- 预计算可供性（4 种呈现）

所谓“GPT-6 Astra 精度弱”，很大程度上出在 harness 而非模型：给它朴素接口，再补上对症且足够短的几何信息就够了。

问题： RUA 分数涨得快，但说不清增益来自哪里

什么是 RUA

机器人使用智能体 Robot-Use Agent

- GPT-6 Astra 看三路相机图像，直接给出末端目标位姿
- 非学习的规划器 CuRobo 生成关节轨迹
- 中间没有 VLA，类比“看屏幕点鼠标”的 computer-use agent

矛盾

“模型弱”还是“ harness 弱”？

- 官方 L3 harness 下， GPT-6 Astra 在精度任务上表现很弱
- 作者只做几处合理修改， Precision 分就接近当时最强 VLA （ DM0.5 ）

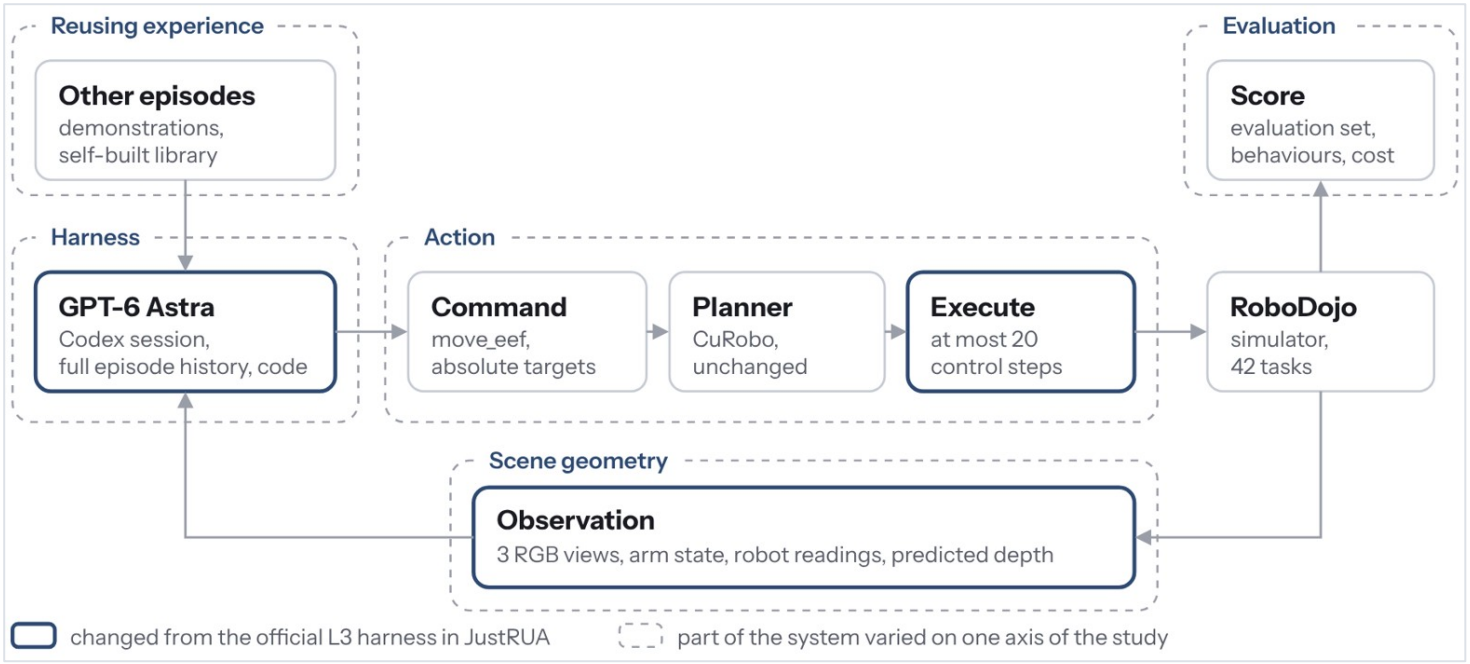
难以比较

各系统同时改了太多东西

- harness 、测试子集、预算都不同
- 常常一次加入深度、可供性、演示、自我改进等多个组件
- 分数更高，却不知道是哪个组件起的作用

本文做法：不复现各个系统，而是把设计拆开，在同一测试床、同一评测集、同一成本口径下逐项对照

一个闭环、四条轴：每次只改一处，看它是否值得保留



蓝色实框是 JustRUA 相对官方 L3 harness 的改动，虚线区域各对应一条实验轴；规划器 CuRobo 与仿真器保持不变。

1 Harness
保留多少历史、推理强度

2 Action
命令格式、每段动作执行多久

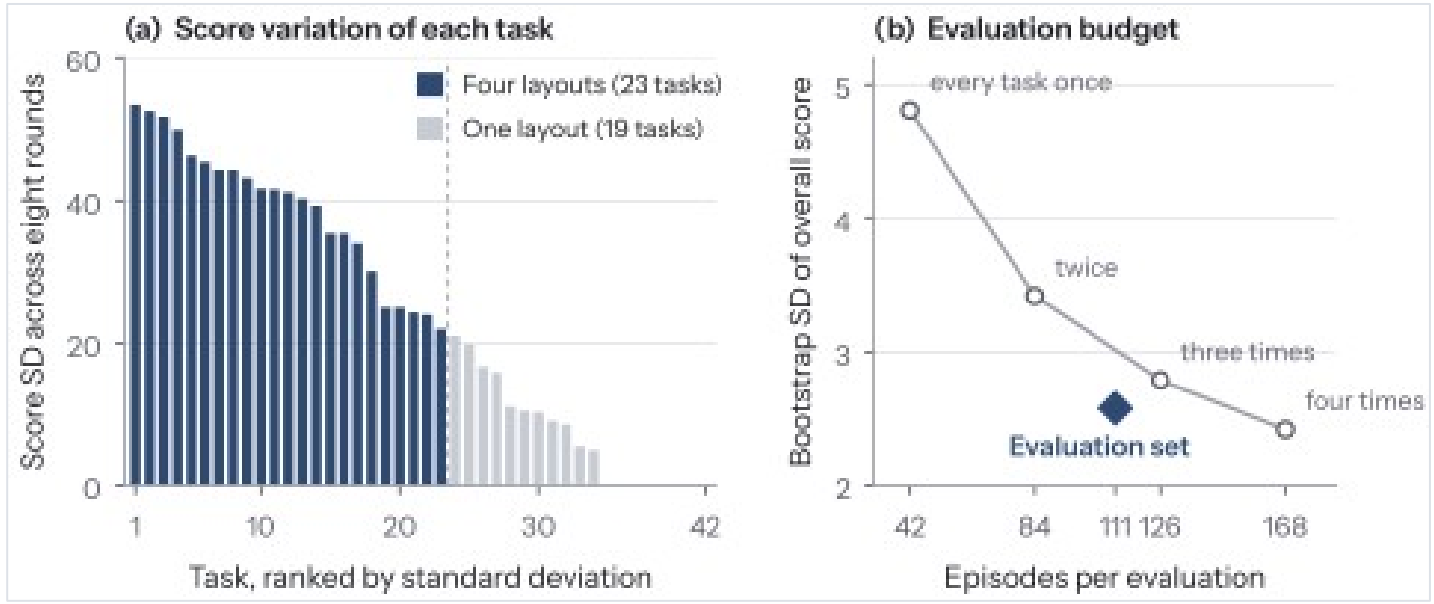
3 Reusing experience
演示 → 文本轨迹 → 自写技能库

4 Scene geometry
读数 → 深度 → 可供性 → 代码求坐标

保留准则（三条同时满足）

- 在评测集上有帮助
- 成本合理
- 不利用被评测任务的信息

111 集的评测集，比每任务跑 3 次（126 集）还稳



(a) 42 个任务在 8 轮中的分数标准差，波动最大的 23 个跑 4 个布局；(b) 不同预算下总分的 bootstrap 标准差。

23×4 + 19×1 = 111

波动大的任务多跑几次，布局固定且所有配置共用

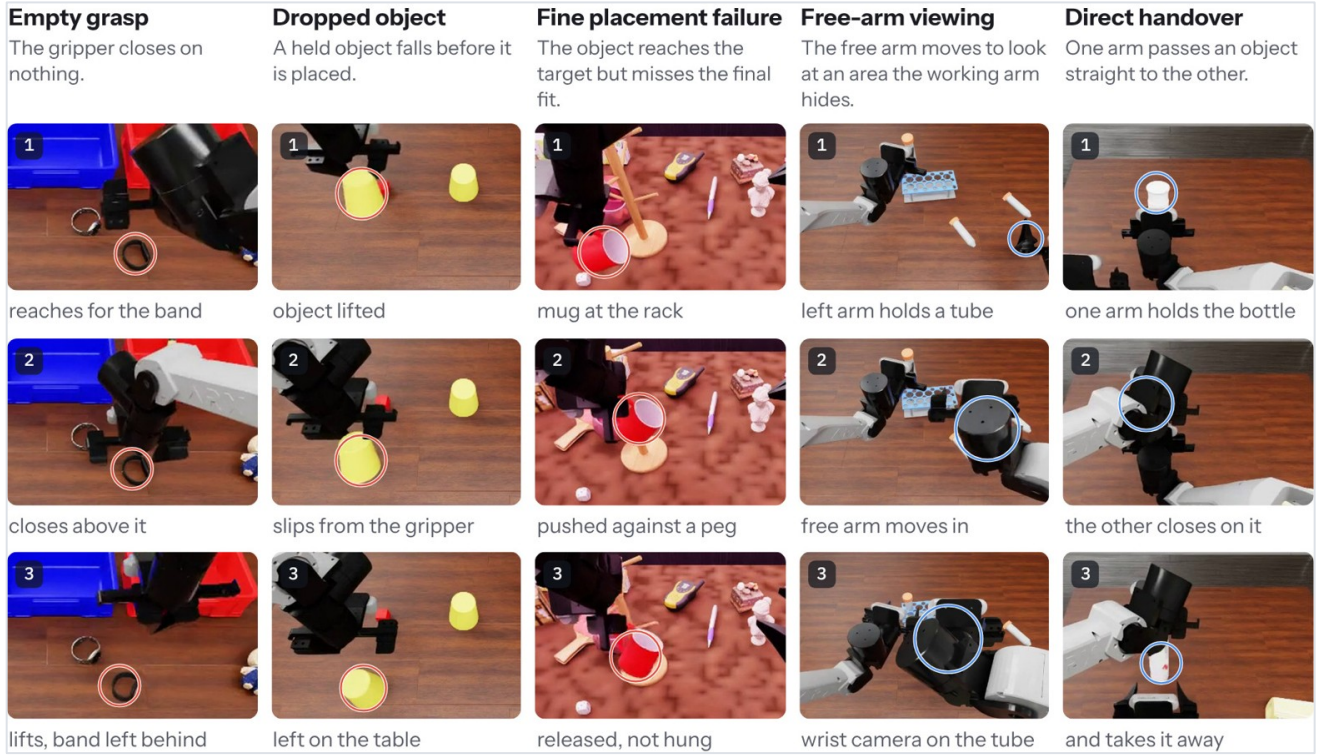
总分的 bootstrap 标准差（medium 推理强度）

评测方式	集数	标准差
每任务 1 次	42	4.8
每任务 3 次	126	≈2.8
本文评测集	**111**	**2.6**
每任务 4 次	168	2.4

29.7 vs 29.0

官方 harness：评测集分数 vs 完整 2,100 集评测

分数之外：逐集审查视频统计行为，并按缓存价计成本



行为审查

独立的 GPT-6.1 Sol 审查首轮 42 集的三路视频和完整动作日志，再做二次核验；只统计有帧证据的事件，计数是下界

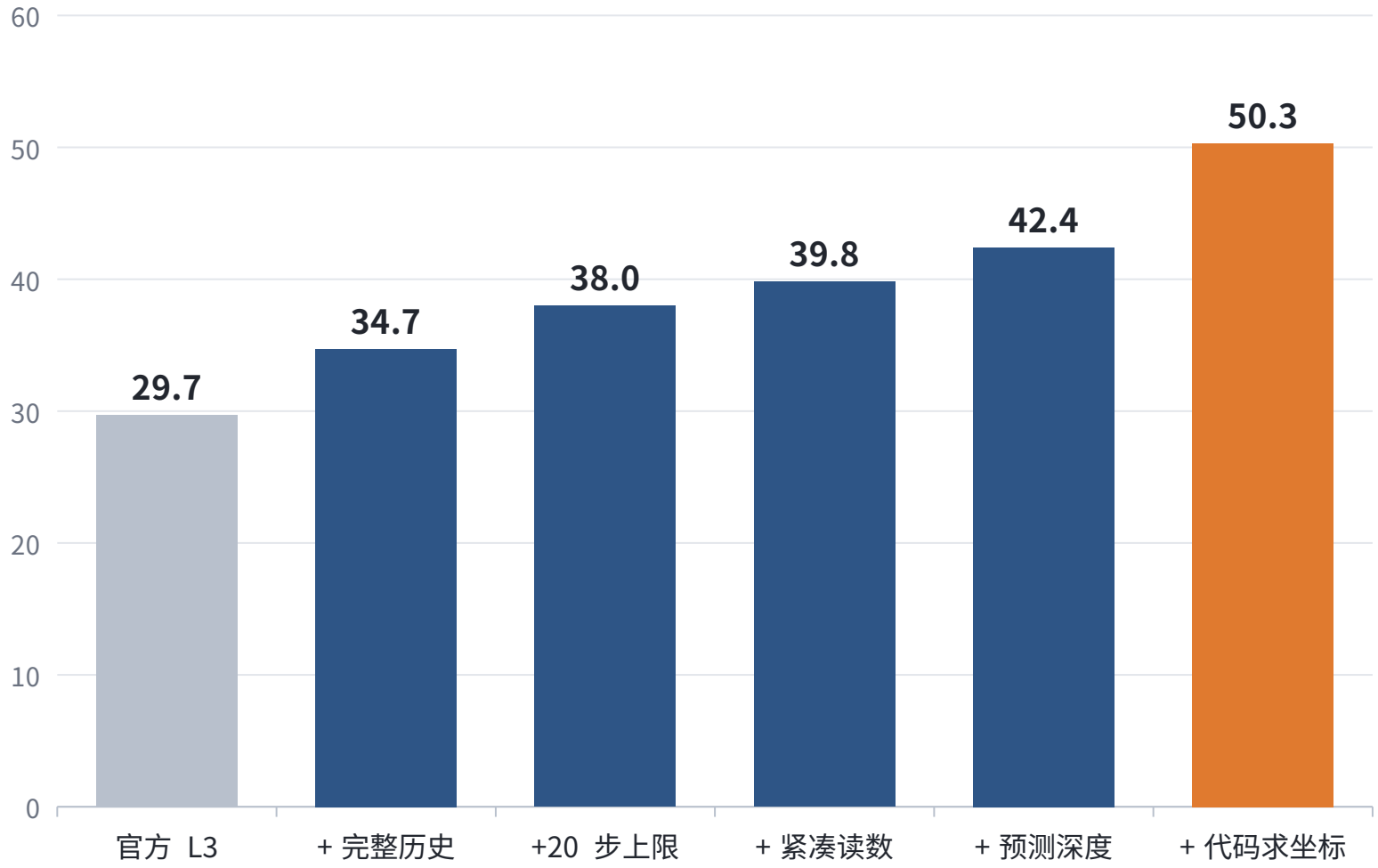
统计哪些行为

空抓、掉落、精细放置失败（失败类）；视角调整、交接（中性）；另计意外碰撞

成本口径

每次决策重发完整历史，一集读几百万输入 token；按“可缓存输入全部命中缓存”计算首轮每集成本下界

沿保留路径累积：最大的两步是早期 harness 和最后的代码求坐标

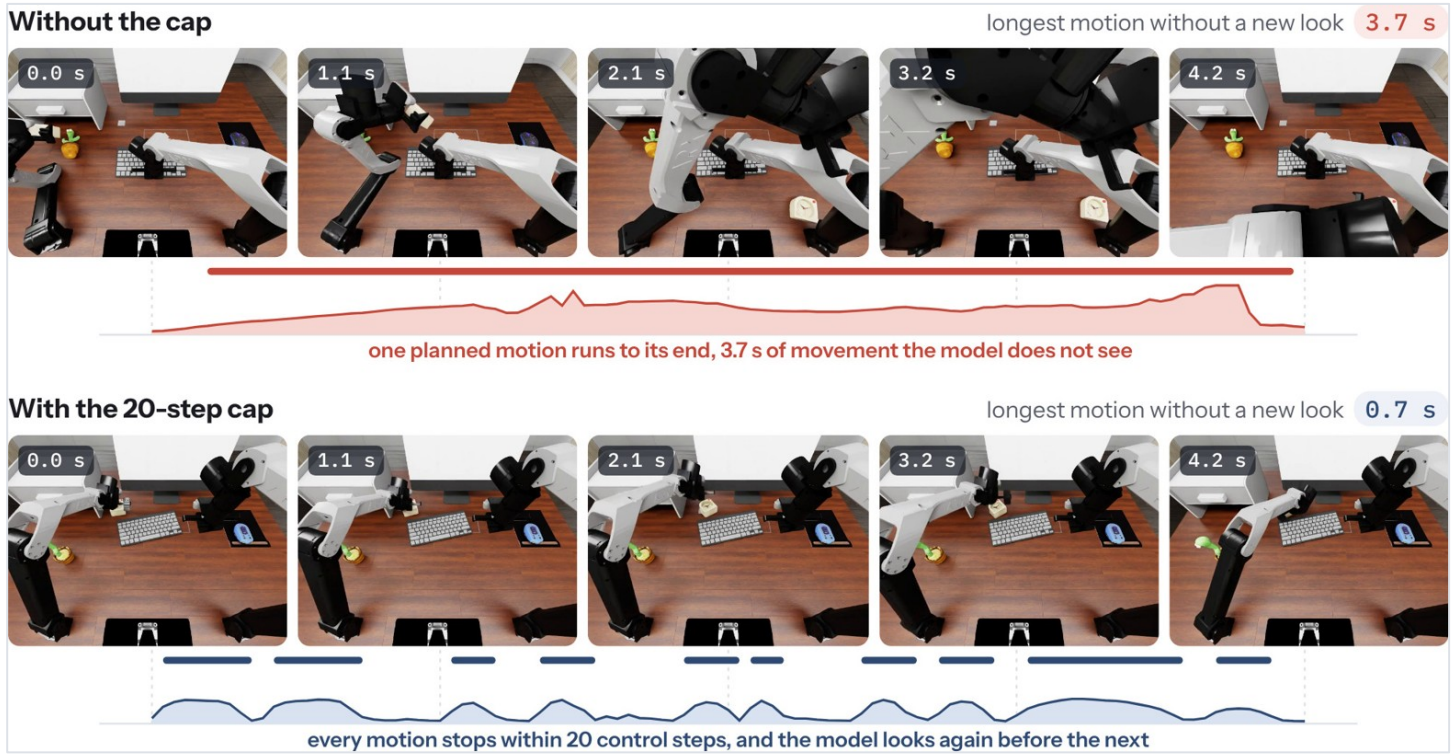


每一步的增量与成本

+5.0	完整历史	$3.76
+3.3	20 步上限	$4.94
+1.8	紧凑读数	$5.51
+2.6	预测深度	$7.74
+7.9	代码求坐标	$12.53

成本为首轮每集下界（全部命中缓存价）。最后一步同时取消了 100 决策上限，两者的贡献无法拆开。

Harness： 保留完整历史 + 每段动作 20 步早停



organize_table 同一段 4.2 秒：无上限时，一次不稳定的规划让手臂横扫桌面挡住相机，模型全程看不到。

3.7 s → 0.7 s

模型“看不到”的最长连续运动

精度组 9.4 → 25.0

20 步上限的增益集中在 Precision 组

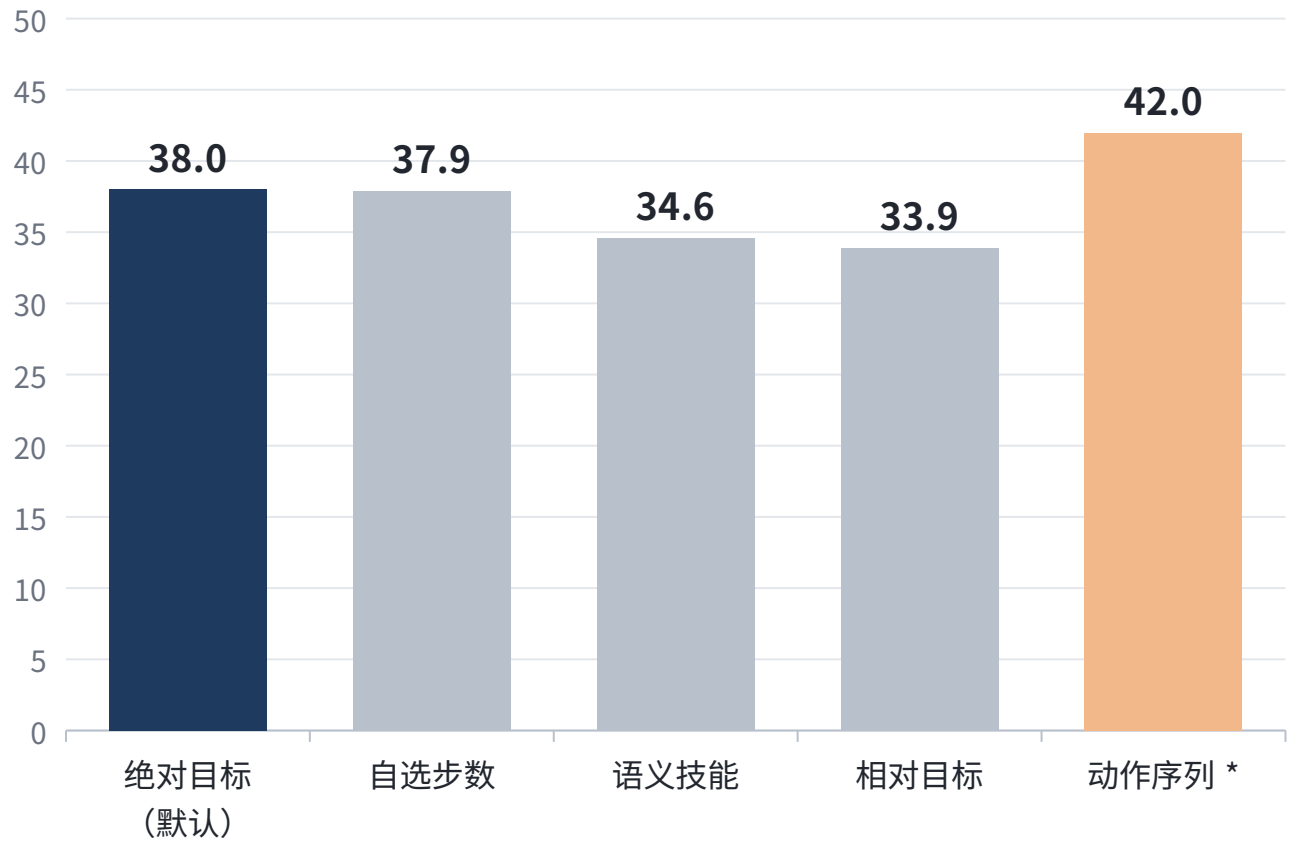
29.7 → 34.7 → 38.0

完整历史，再加 20 步上限（评测集总分）

截断但不重采样，剩余轨迹直接丢弃；反馈仍按原目标报告误差和实际执行步数。

只迁到 Codex 而不保留历史，本身并无帮助。

更“聪明”的动作接口，没有胜过朴素的绝对目标



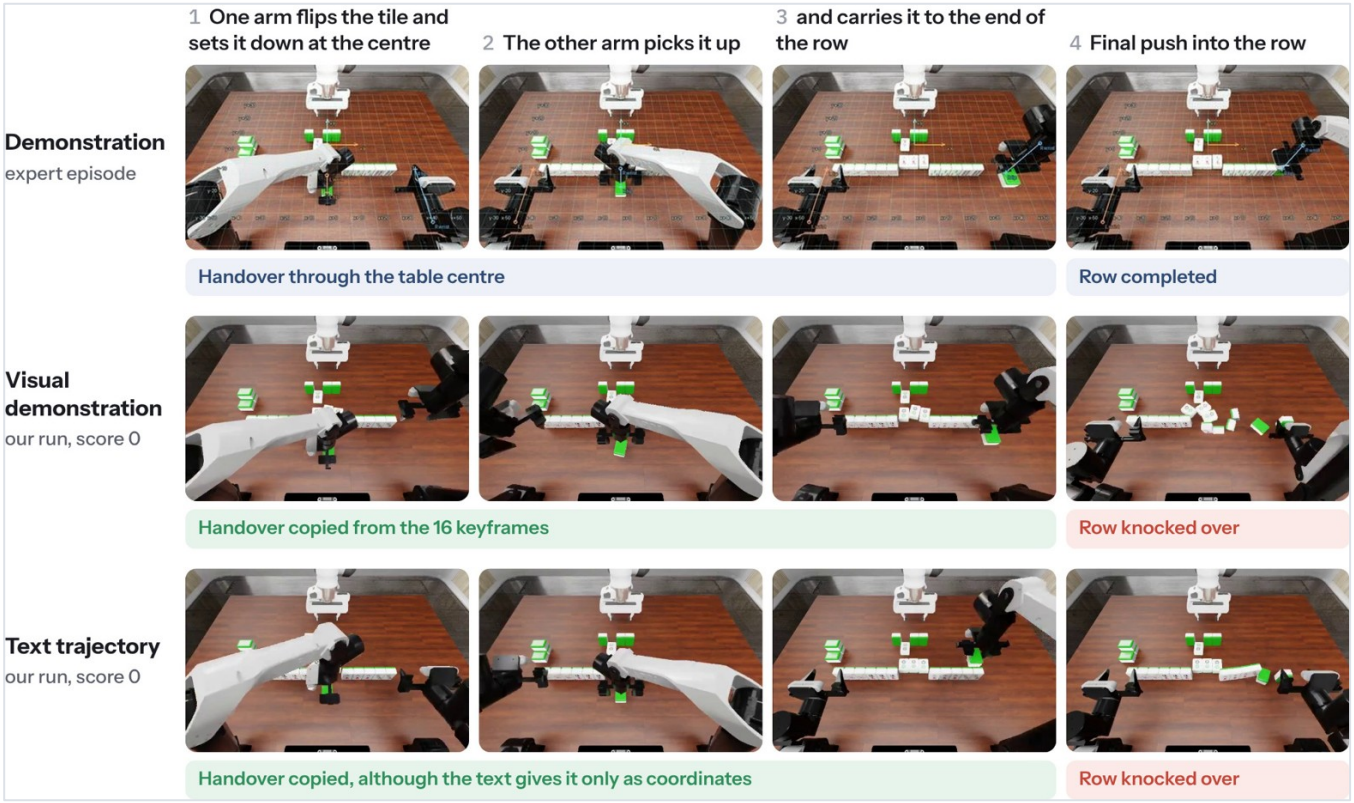
* 动作序列的增益主要来自 6 个原本耗尽 100 决策预算的 episode；若它们保持原分数，总分只有 39.4，所以 JustRUA 选择直接取消预算。

首轮 42 集的行为计数

行为	绝对	相对	语义	序列
空抓	47	54	47	38
掉落	26	30	35	27
视角调整	59	54	211	45
手臂撞物体	7	20	12	1

- **相对目标：**撞物体约 3 倍，可能是输入全为世界坐标，多一步减法
- **语义技能：**视角调整多 3 倍以上、掉落更多，改变了“做什么”而非“做多好”
- **推理强度：** medium 最好； low 精度组仅 5.3

复用经验改变了行为，却没换来分数：不是免费午餐



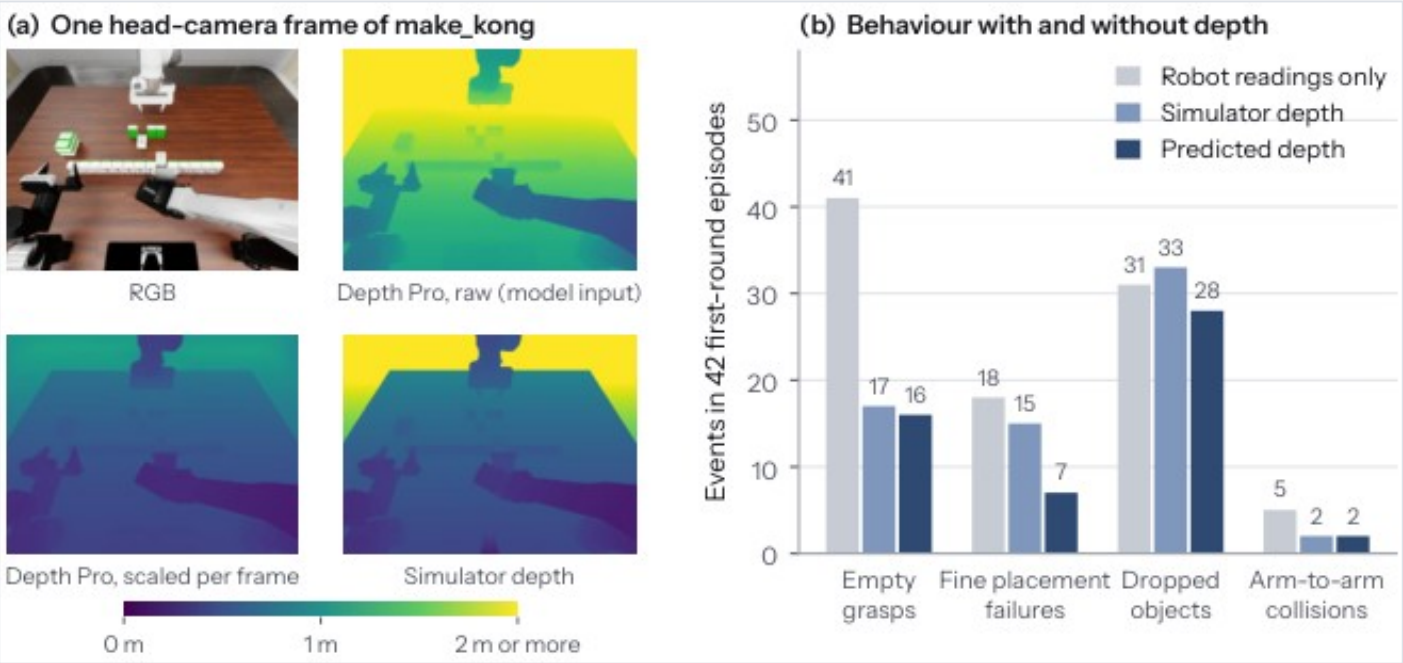
make_kong：两种演示都让模型学会经桌面中央交接，但最后推入一行时都把整行撞倒，得 0 分。

配置	评测集	成本 $
默认测试床	38.0	4.94
视觉演示	39.9	5.11
文本轨迹	35.2	5.84
自建技能库	37.0	5.43

- 演示：交接更多，掉落 26 → 20 / 17
- 技能库：空抓 47 → 33，臂间碰撞 6 → 0
- 修好一个环节，episode 常在别处失败

经验来自被评测的任务，分数与标准评测不可比，JustRUA 一律不保留

预测深度误差几十厘米，却和仿真深度一样有用



(a) 头部相机 RGB 、 Depth Pro 原始预测（模型实际输入）、逐帧缩放后结果与仿真深度； (b) 三种设置下的行为计数。

Depth Pro 预测精度（头部相机）

设置	MAE	r
原始预测（模型输入）	**73.7 cm**	**0.993**
真实焦距	40.1 cm	0.993
逐帧缩放	7.3 cm	0.993

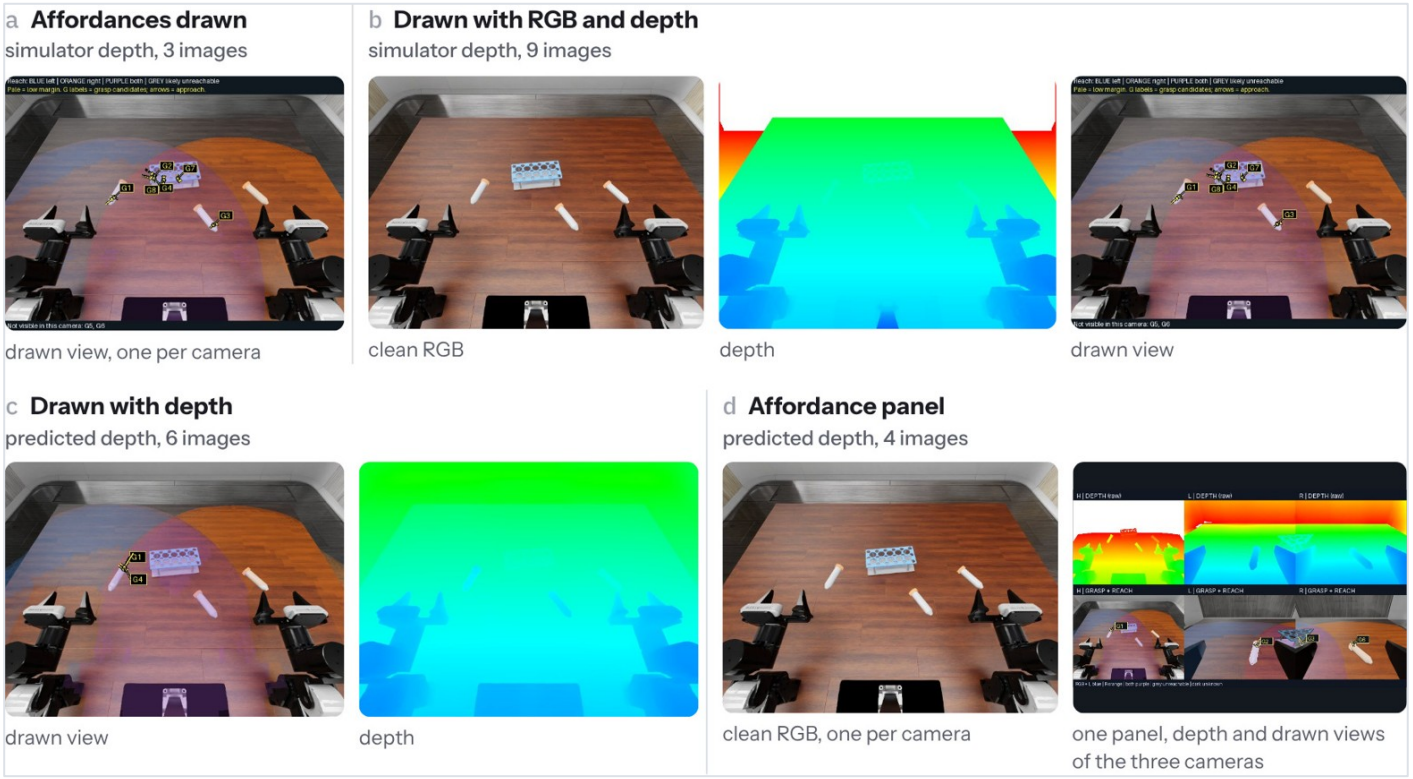
41 → 16
首轮空抓次数

18 → 7
精细放置失败

总分： 紧凑读数 39.8 → 预测深度 42.4 （仿真深度仅作参考， 46.0 ，榜单不接受）

启示： 深度的“相对顺序”对了就够用，这项增益不依赖特权信息

预计算可供性没有帮助：偶尔有用的信息，不值得每步付费

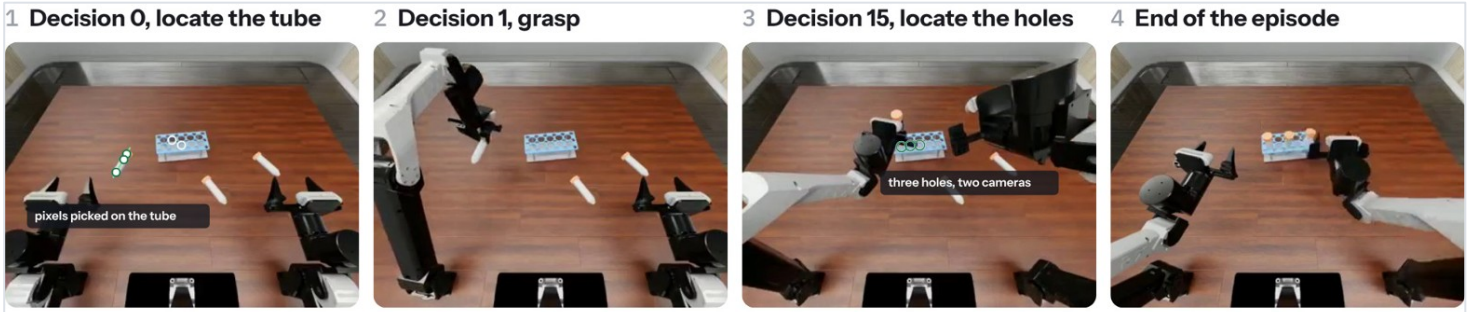


配置	评测集	空抓
仿真深度	46.0	17
+ 标注（仿真深度）	36.7	17
预测深度	**42.4**	**16**
+ 标注（预测深度）	30.9	39
+ 汇总面板（预测）	42.1	34

- 与同源深度相比，四种呈现都没有抓得更稳、分数更高
- 可达性只在交接时有用，而交接一直很少
- 抓取候选只在开始抓取的决策有用，成本却每次观测都要付

四种呈现：标注画在视图上 / 加回 RGB 与深度 / 预测深度 + 标注 / 汇总面板（GraspGen 抓取候选 + IK 可达区域）

让模型写代码从像素解出 3D 坐标：分数升到 50.3



The model casts rays from pixels of the left tube through the head camera onto the table plane.

The left gripper goes to the solved point, **1.2 mm from the tube axis**, closes and lifts the tube.

Three holes of the rack and the tube tip are triangulated from the head and right wrist cameras.

All three tubes stand in the rack, **score 100**. A later decision reuses a solved hole, 0.4 mm off.

CODE THE MODEL RUNS AT DECISION 0

```
import numpy as np
for u,v in [(220,202),(201,235),(215,212),(302,177),(319,186)]:
 d=np.array([(u-320)/288.1325,-.86603*(v-240)/288.1325+.5,
             -.5*(v-240)/288.1325-.86603])
 p=np.array([0,-.41,1.308]); q=p+d*((.782-p[2])/d[2]); print(u,v,q)
```

OUTPUT

```
220 202 [-0.22816849 -0.00619792  0.782]
201 235 [-0.2533848  -0.09402191  0.782]
215 212 [-0.23449104 -0.03411066  0.782]
302 177 [-0.04342494  0.0691849   0.782]
319 186 [-0.00236371  0.04107148  0.782]

→ decision 1: left_x -0.232, left_y -0.024
```

93.2%

代码运行用于几何计算（123/132）

40 → 100

insert_tubes 同一布局，决策 65 → 57

长程组 31.6 → 52.2

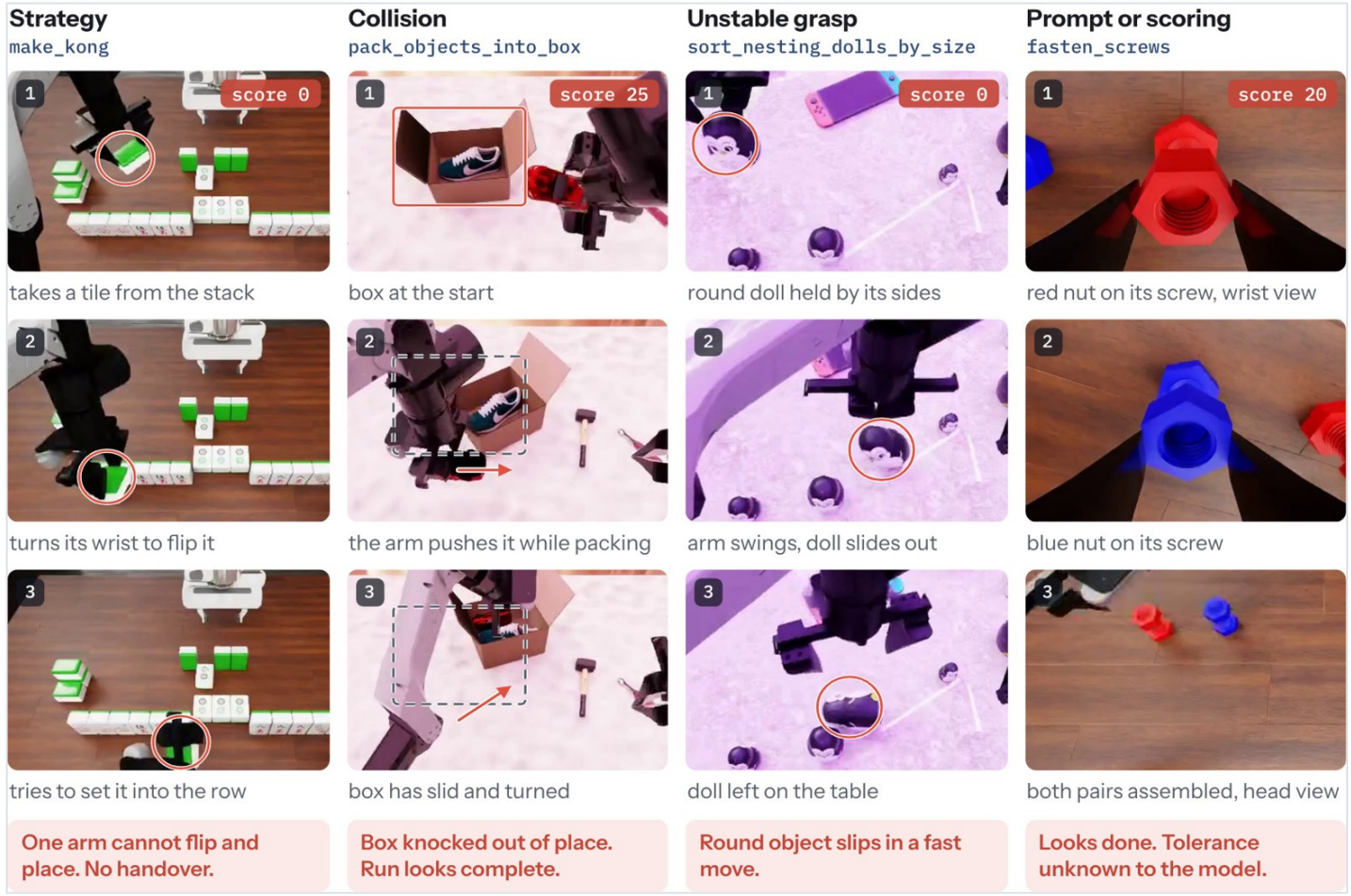
增益最大； Memory 组 55.6 → 70.8

$12.53 / 集

最贵的一步；同时取消 100 决策上限

insert_tubes：像素射线与桌面求交、两路相机三角化孔位（距离为代码输出与后续命令之差，非与真值之差）

剩余失败多是逐任务的长尾： 21 / 42 个首轮任务低于 50 分



8 **策略（任务数）**

计划不适合机械臂： make_kong 不交接、正向倾倒

6 **碰撞（任务数）**

误判手臂或持物尺寸：轻盒子被推走

6 **抓取不稳（任务数）**

圆形、不规则物体滑落：套娃、硬币

5 **提示 / 评分（任务数）**

看似完成，但模型不知道评分公差：fasten_screws

按评分条件改写提示：首轮 54.0 → 55.6 （利用了评分器，不计入 JustRUA ）

再提分要逐任务试错，已接近在测试任务上调参

两条主线：朴素接口就够，信息要对症且足够短

GPT-6 Astra 用朴素接口就能做好

世界坐标绝对目标、原始传感器值、未校准深度都够用；重组动作或预消化观测的中间层，主要增加的是成本。

例：相对目标 / 语义技能 / 可供性 均未胜出

有效信息：对症 + 足够短

针对模型高频犯的错（空抓、物体定位不准），而且足够紧凑，每次决策都付得起。

例：完整 JSON 读数让 38% 首轮 episode 触发上下文压缩

Takeaway 1

精简 harness 走很远

29.7 → 38.0

Takeaway 2

复用经验非免费午餐

无明显增益且更贵

Takeaway 3

场景几何帮助最大

空抓 41 → 16

Takeaway 4

剩余失败是长尾

多为单任务特有

局限与评价：一份扎实的“减法”研究，但细小差异需谨慎解读

作者自述的局限

- 只在仿真、单一 benchmark 、单一模型上实验，每个配置在评测集上只跑一次
- 路径式消融而非全因子，只检验“在已保留设计之上是否有用”
- 经验类实验利用了被测任务；自我改进只跑了 5 轮，每任务练习 1 次
- 评测集由作者自己默认测试床的历史挑选，Generalization 组随机布局占 75%（官方为 50%），不是榜单分数

解读者观察

- 评测集标准差约 2.6： 39.8 / 39.9 / 42.1 / 42.4 等配置难以区分，可靠的是 29.7 → 38.0 与 42.4 → 50.3 两段
- 最后一步把“代码求坐标”和“取消决策上限”捆在一起，而动作序列实验已表明预算本身影响很大
- “简单”不等于便宜：每集成本下界从约 $3.8 涨到 $12.5
- 价值在于纠正对增益来源的误读，并给出一个强而简单的基线